GROCERY STORE MANAGEMENT & WHATSAPP ORDERING SYSTEM

Architecture-Driven Implementation Baseline

Version 3.0 | Revised architecture and implementation specification

Prepared: 24 October 2026

Purpose

A practical, testable baseline for building a secure multi-tenant grocery platform with store management, inventory, order review, WhatsApp ordering, invoices, payment links, notifications and reporting.

Important engineering statement

No architecture document can guarantee a “100% correct” or bug-free implementation. This document defines decisions, invariants, workflows, failure handling and evidence required to demonstrate correctness and production readiness.

Status: proposed implementation baseline. Provider-specific, tax, hosting and business-policy decisions must be approved and recorded before production integration.

Document Overview

- 1. Executive summary and design principles
- 2. Scope, actors and business assumptions
- 3. Architecture and service boundaries
- 4. Data ownership and tenancy
- 5. Domain model and state machines
- 6. Critical workflows
- 7. Inventory correctness and distributed transactions
- 8. Payments, invoices, tax and refunds
- 9. WhatsApp ordering and notification reliability
- 10. Security and privacy
- 11. API, events and integration contracts
- 12. Data design and migrations
- 13. Observability and operations
- 14. Testing and acceptance criteria
- 15. Deployment, backup and recovery
- 16. Phased implementation plan
- 17. Risks and architecture decision records
- 18. Definition of done

1. Executive Summary and Design Principles

The system supports multiple independent grocery stores. Each store manages its catalogue, pricing, inventory, customers, orders, invoices, staff permissions and reports. Customers can order through the approved web flow and supported WhatsApp conversation. Store staff review the requested order, confirm final quantities and price, and the platform then coordinates inventory reservation, billing, payment-link creation and customer communication.

The design prioritizes correctness and recoverability over infrastructure complexity. Use synchronous APIs for immediate decisions, durable events for asynchronous work, local database transactions for local invariants, and explicit workflow state for cross-service operations.

- **Authoritative state:** each domain service owns its write data; cache, browser state, messages and provider redirects are not authoritative.
- **At-least-once reality:** assume requests, events and webhooks may be duplicated, delayed, reordered or lost before acknowledgement.
- **Idempotent effects:** repeated commands must not duplicate orders, reservations, invoices, charges, refunds or logical notifications.
- **Tenant isolation:** tenant identity comes from authenticated identity or a verified provider mapping, never from an untrusted request field.
- **Observable workflows:** every pending state has a timestamp, timeout/reconciliation rule, metrics and an operator recovery path.
- **Evidence over claims:** production readiness requires passing tests, security review, staging evidence and restore drills.

2. Scope, Actors and Business Assumptions

Core release scope

- Store registration and profile; store owner and staff roles.
- Product catalogue with categories, subcategories, brands, variants/SKUs, images, units and active status.

- Price management and effective-dated price changes where needed.
- Inventory balances, stock movements, adjustments, reservations, release and low-stock/out-of-stock alerts.
- Customer identification and controlled access to customer order history.
- Web browsing and ordering; supported WhatsApp ordering conversation and order-status notifications.
- Shopkeeper review: confirm, reject, request changes, or propose alternatives.
- Itemized immutable invoice/bill snapshot after confirmation, subject to approved tax rules.
- Payment-link creation and verified payment-status updates through a selected gateway.
- Order fulfilment/status tracking, basic sales and inventory reporting, audit records, email delivery when an address is available.

Explicit pre-build decisions

Decision	Required resolution
Payment provider	Select provider, supported payment-link/refund operations, idempotency support, webhook signature scheme, sandbox and reconciliation API.
Tax/invoicing	Confirm jurisdiction, invoice numbering, tax-inclusive/exclusive pricing, rounding, credit notes, retention and accountant review.
Fulfilment	Pickup/delivery, delivery fee, service area, time windows, substitutions and who may mark an order completed.
Order policy	Review timeout, cancellation cutoff, reservation timing/expiry, price changes, partial fulfilment and unavailable items.
Customer identity	Phone normalization, web-to-WhatsApp account linking, verification and duplicate customer policy.
WhatsApp	Cloud API account, business-number-to-tenant mapping, templates, consent/opt-out, conversation window and rate limits.
Hosting/SLO	Hosting target, initial traffic profile, latency/error objectives, RPO/RTO, data region and operational owner.

These are genuine business or environment choices, not safe assumptions to invent. Record each approved decision as an Architecture Decision Record (ADR).

3. Architecture and Service Boundaries

Recommended logical flow: browser / WhatsApp provider → edge controls and API Gateway → domain APIs. Services publish durable domain events through a transactional outbox to RabbitMQ. Notification and Reporting consume events. Payment and WhatsApp provider callbacks enter dedicated verified endpoints and are routed to the owning service.

Component	Owns / responsibilities	Boundary rule
API Gateway	Routing, request IDs, coarse rate limits, request size, timeouts and consistent edge errors.	Not the only authorization layer; no business-data writes.
Identity	Users, credentials, sessions, roles and account state.	Provides verified identity and permissions; supports revocation.
Catalog	Products, variants, SKU, units, product metadata and price versions.	Order stores immutable item snapshots; client prices are ignored.
Inventory	Stock balances, reservation records, movement ledger and stock alerts.	Only service allowed to mutate inventory; atomic all-or-nothing reservation.
Order	Order lifecycle, requested/approved quantities, customer/store references and workflow coordination.	Owns order state and saga; never claims confirmed stock without reservation success.
Billing	Immutable invoice snapshots, numbering and document generation.	Invoice source is the approved order snapshot and tax policy.
Payment	Payment attempts, provider references, payment links, webhook inbox and refunds.	Only verified provider evidence/reconciliation changes payment state.

Component	Owns / responsibilities	Boundary rule
Notification	WhatsApp/email templates, send attempts, delivery receipts and retries.	Asynchronous; notification failure must not erase a valid order.
Reporting	Read projections and report APIs.	Consumes versioned events; no direct reads from other services' private tables.

Deployment and service count

Keep clear logical ownership from the beginning. Independently deploy a service only when its boundary justifies the operational cost. Billing may initially be a module within Order if the team cannot yet operate a separate service, but its data and interfaces must remain clearly owned. If the project requires microservices, deploy the agreed services independently while avoiding circular synchronous calls.

For a small first deployment, services may share a PostgreSQL server but must use separate databases or schemas and least-privilege roles. Never share write tables or create cross-service foreign keys. Separate database servers can be introduced when justified.

4. Data Ownership and Tenant Isolation

- Every tenant-owned row has a tenant/store identity and is scoped using identity established by verified authentication or a trusted internal service context.
- Never authorize access from a tenant_id supplied by a browser, URL, header or message alone.
- Every service validates both role permission and resource ownership on every read, write, export and document download.
- WhatsApp inbound events resolve the tenant only through verified provider business-account/phone-number mapping.
- Use database constraints and tenant-scoped unique keys. Consider PostgreSQL Row-Level Security as defense in depth, with carefully tested connection context and privileged-role bypass controls.
- Service-to-service calls use authenticated service identities, least privilege, short-lived credentials where possible, and documented authorization scopes.
- Audit store membership changes, role changes, inventory adjustments, order revisions, refunds and sensitive state transitions.

Tenant isolation acceptance test

Create Store A and Store B, each with distinct users, products, stock, customers, orders, invoices, payments and reports. Attempt cross-tenant reads and writes using changed resource IDs, forged tenant fields, direct service calls, exports, invoice URLs and webhook mappings. Every unauthorized attempt must be denied and safely logged. Repeat as automated regression tests.

5. Domain Model and State Machines

Minimum domain entities

Domain	Entities and invariants
Identity	User, Store, StoreMembership, Role/Permission, Session/RefreshToken. Membership is the explicit relationship between user and store.
Catalog	Product, ProductVariant/SKU, Category, Brand, UnitOfMeasure, PriceVersion, ProductImage.
Inventory	StockBalance, StockMovement, Reservation, ReservationItem. Movements are append-only; balances are reconciled against the ledger.
Order	Order, OrderItemSnapshot, OrderRevision, Fulfilment, CancellationRequest, IdempotencyRecord.
Billing	Invoice, InvoiceLineSnapshot, InvoiceSequence, CreditNote where required, generated document reference.
Payment	PaymentAttempt, ProviderEventInbox, Refund, ReconciliationRecord.
Messaging	InboundMessage, OutboxEvent, ProcessedEvent/Inbox, NotificationAttempt, DeliveryReceipt.
Governance	AuditRecord, Migration, operational event and reconciliation run metadata.

Order state

RECEIVED → AWAITING_SHOPKEEPER_REVIEW → CHANGE_REQUESTED (optional) → CONFIRMATION_PENDING_RESERVATION → CONFIRMED → PREPARING → READY_FOR_PICKUP / DISPATCHED → COMPLETED / DELIVERED. REJECTED and CANCELLED are terminal business outcomes subject to explicit policy. Reservation failure returns to a review/revision path or a defined failed state; it must never silently confirm the order.

Implement allowed transitions in one domain policy/use-case layer and enforce them transactionally. Every transition records actor/source, timestamp, reason, previous state and new state. State names and transitions must be consistent across API docs, events, tests and UI.

Payment and refund state

Payment and order fulfilment are separate state machines. Model payment attempts as CREATED → PENDING_PROVIDER → PAID / FAILED / EXPIRED / UNKNOWN, adjusted to the selected provider's semantics. UNKNOWN means the outcome is uncertain and requires reconciliation, not that the payment failed. Refunds are separate records with PENDING → SUCCEEDED / FAILED states. A refund failure does not erase the original successful payment.

Inventory reservation state

Reservation states should include PENDING, RESERVED, RELEASE_PENDING/RELEASED, EXPIRED and FAILED as appropriate. State transitions must be idempotent. A reservation identifier is the stable key for release and compensation.

6. Critical Workflows

A. Shopkeeper-reviewed order

- Accept order intent with an idempotency key; validate authenticated customer/store context.
- Fetch authoritative catalogue/price data; reject client-supplied totals. Store requested product and quantity details.
- Create order in AWAITING_SHOPKEEPER_REVIEW and persist an outbox event in the same Order database transaction.
- Shopkeeper reviews quantities, substitutions and price. A revision creates an auditable new version; customer confirmation is required if policy demands it.
- When final terms are accepted, Order starts a durable reservation workflow with a unique workflow/reservation ID.
- Inventory atomically reserves every requested item or none. It records the outcome and emits a durable event.
- Order moves to CONFIRMED only after verified reservation success and final approval. On failure, it returns to review or fails according to policy.
- Billing creates one immutable invoice/bill snapshot from the approved order revision. Payment creates a link for exactly the approved payable amount if the chosen policy requires prepayment.

- Notification workers deliver confirmation, invoice and payment link. Delivery failure is retried independently and never reverses a valid order.

B. Cross-service recovery

Use a saga/workflow record in Order with a durable step, correlation ID, attempt count, last error category and next action. A process crash resumes from persisted state. Every command is idempotent, each remote call has a timeout, and compensation has its own retry and monitoring. Never hold a database transaction open while waiting for a remote service.

C. Cancellation

- Atomically validate whether cancellation is allowed for the current order state and persist the cancellation request.
- Release a reservation using its stable ID and an idempotent Inventory operation.
- If payment is successful, initiate a provider-supported refund or a documented manual-review path; do not mark the payment as unpaid.
- Resolve races between payment completion and cancellation with a documented state-transition policy, provider reconciliation and an auditable final outcome.
- Notify the customer only after the relevant state has been durably recorded; notification itself may retry.

7. Inventory Correctness and Distributed Transactions

Inventory is the authority for stock availability. A check-then-update sequence without concurrency control is prohibited.

- Use an atomic conditional update or row lock inside an Inventory-local transaction; verify affected row counts.
- For multi-item reservations, lock/validate rows in a stable SKU order and commit all-or-nothing in the Inventory database.
- Enforce non-negative available stock and appropriate unique constraints in the database.
- Represent available, reserved and on-hand quantities consistently; document the equation, for example on_hand = available + reserved, with explicit treatment of damaged/quarantined stock.
- Record every receipt, sale, reservation, release, adjustment, shrinkage and return in an immutable stock movement ledger.
- Use optimistic version checks or row locks for staff adjustments to prevent lost updates.
- Define reservation expiry and an expiry worker. Expiry and manual release must be idempotent and safe under races.
- Run a reconciliation job to compare balances, reservations and movements, report discrepancies and support controlled repair.

Required concurrency test

Given 10 available units and 100 simultaneous reservation attempts for one unit each, no more than 10 reservations succeed and the final available quantity is never negative. Repeat with multi-item orders, duplicate requests, cancellation-versus-confirmation races, worker crashes and database retries.

8. Payments, Invoices, Tax and Refunds

- Choose and document the provider before implementing its adapter. Use provider sandbox credentials for development and staging.
- Use a local payment attempt and a stable idempotency key; persist the request/result reference before or immediately around provider interaction using a recoverable state machine.
- Verify webhook signatures according to provider instructions, using the raw request body if required. Store provider event IDs under a unique constraint before applying effects.
- Handle duplicate, delayed and out-of-order webhooks. Apply state transitions only when valid and monotonic under the provider's semantics.
- On timeout, mark the result UNKNOWN/PENDING_RECONCILIATION and query the provider before creating another logical payment.
- Create payment links only for an approved invoice/order revision; amount and currency must be server-derived and cross-checked.

- Use integer minor units or PostgreSQL NUMERIC for money, never binary floating point for authoritative amounts.
- Define taxes, discounts, delivery fees, rounding, currency, invoice numbering, credit notes and document retention with a qualified business/tax owner.
- Store immutable invoice line snapshots and the calculation inputs. Product price changes must not rewrite historic invoices.
- Refunds must be idempotent, permission-controlled and auditable. Reconcile local payment/refund records against provider records on a schedule.

Do not launch real payments until the provider's webhook, idempotency, timeout, duplicate-event, refund and reconciliation scenarios pass in a sandbox and staging environment.

9. Grocery Units, Variants and Pricing

- Represent sellable items as explicit variants/SKUs where package size, brand, weight, barcode or price differs.
- Define a canonical unit of measure and supported conversions (for example kg/g or litre/ml). Do not use free-form unit text in stock arithmetic.
- Decide whether fractional quantities are permitted per SKU; validate precision and minimum increments (for example 0.25 kg if supported).
- Keep stock quantity precision separate from monetary precision. Define quantity rounding and line-total rounding.
- Store a price version or approved price snapshot on each order item. Record changes in an auditable price history.
- Define substitutions, partial fulfilment, weighed-at-pickup differences, out-of-stock handling and final customer approval.

10. WhatsApp Ordering and Notification Reliability

- Use the official WhatsApp Business Platform Cloud API unless an approved alternative is explicitly recorded.
- Verify webhook authentication/signature, map the business phone-number/account ID to one tenant, and reject unknown or invalid mappings.
- Persist provider message/event IDs with unique constraints before creating business effects. Acknowledge inbound callbacks quickly after durable acceptance.
- Model the conversation state explicitly: browsing/intent, cart review, missing information, shopkeeper review, confirmation, payment and completion.
- Ask clarifying questions for ambiguous messages. Never convert a vague message into a submitted order without confirmation.
- Apply the same Order/Catalog/Inventory APIs and business rules used by web orders.
- Handle message templates, customer consent/opt-out, provider rate limits, delivery receipts, retryable versus permanent errors and conversation-window rules.
- Use an outbound notification queue and persist attempts, provider message IDs, delivery state and retry schedule. Suppress duplicate logical sends.
- Keep email optional when no address exists, but expose delivery status to staff and provide a manual recovery path for failed critical messages.

11. Security and Privacy

Control area	Minimum implementation requirement
Authentication	Argon2id or appropriately configured bcrypt; secure account recovery; login throttling; session expiry/revocation.
Browser sessions	Prefer Secure, HttpOnly, SameSite cookies with CSRF defenses when cookie-authenticated; define token storage/refresh/revocation if using tokens.
Authorization	Server-side role + tenant + resource ownership checks on every operation; test BOLA and privilege escalation.
Input and output	Runtime validation (e.g. Zod); parameterized queries; safe output rendering; upload size/type validation; prevent SSRF if fetching user-controlled URLs.
Network	HTTPS; appropriate HSTS and security headers; strict CORS allowlist; request limits; private internal service endpoints.
Secrets	Secret manager or protected environment injection; no secrets in source control, logs, images or frontend bundles; rotation runbook.
Infrastructure	Least-privilege service/database roles; non-root containers where practical; patching, dependency scanning and image scanning.
Audit and privacy	Audit sensitive actions; minimize personal data; define consent, retention, deletion, export access and backup-retention policy.
Webhooks	Provider-specific signature verification, deduplication, replay handling and safe error responses.
Operations	Security review and remediation of critical/high findings before launch; access-controlled logs and exports.

12. API, Events and Integration Contracts

API rules

- Use versioned routes such as /api/v1 and publish OpenAPI specs with request/response examples.
- Use consistent problem-details errors, request IDs, validation details and correct HTTP status codes.
- Set explicit request, connection and dependency timeouts; bound pagination, uploads, sorting and filter complexity.
- Accept Idempotency-Key on operations that can create duplicate effects. Persist actor/tenant scope, request hash, status and result reference; reject reuse with a different request.
- Use authenticated service-to-service communication and validate payloads at every trust boundary.
- Do not automatically retry non-idempotent calls without an idempotency contract.

Event contract

Every event includes event_id, event_type, schema_version, occurred_at, producer, aggregate_id, tenant/store context where needed, correlation/trace ID and a documented payload. Do not publish secrets or unnecessary personal data. Use compatible schema evolution and contract tests.

Transactional outbox/inbox

Write business state and outbox row in the same local transaction. A publisher retries until RabbitMQ confirms durable acceptance, then marks the outbox row published. Consumers deduplicate event IDs with a durable inbox/processed-event record and commit business effects before acknowledging. Assume at-least-once delivery; exactly-once end-to-end effects are not assumed.

Event ordering

Where order matters, include aggregate version/sequence and reject or defer stale transitions. Do not assume the broker globally preserves business ordering across multiple queues, retries or consumers.

13. Data Design and Migrations

- Use PostgreSQL constraints for uniqueness, foreign keys within a service, valid amounts, quantity precision and non-negative stock.
- Use tenant-scoped unique constraints where identifiers are only unique within a store.
- Index measured access paths: tenant + status + created_at, SKU, customer/order history, provider reference, event state and outbox backlog.
- Use stable sort keys for pagination and avoid unbounded exports or queries.
- Choose Prisma or a consistent typed SQL layer; use explicit SQL for locking or query plans where needed.
- Keep migrations version-controlled, reviewed and tested against representative data. Use expand/migrate/contract when old and new service versions may overlap.
- Budget database connections across every service replica; set pool limits, statement timeouts and transaction timeouts.
- Keep transactions short and retry only known transient failures with bounded attempts and jitter.
- Document how stock balances are reconciled with the movement ledger and how payment/invoice totals are independently checked.

14. Observability and Operations

Signal	Examples / response
Service health	Liveness, readiness, startup/config validation, request rate, 5xx, p50/p95/p99 latency.
Database	Pool saturation, slow queries, deadlocks, connection errors, migration status.
Events	Outbox age, publish failures, queue depth/oldest age, retries, dead-letter count, consumer lag.
Business workflow	Orders stuck in review/confirmation, expired reservations, payment UNKNOWN age, refund failures, notification failures.
Security	Repeated login failures, denied cross-tenant access, invalid webhooks, rate-limit triggers, suspicious adjustments.
Recovery	Backup job status, restore drill status, reconciliation discrepancy count, RPO/RTO results.

Use structured JSON logs and correlation IDs across REST calls, events and provider calls. Redact tokens, passwords, session IDs and payment-sensitive data. Do not use high-cardinality personal IDs as metric labels. Every actionable alert needs an owner, runbook, escalation and recovery verification.

15. Testing and Acceptance Criteria

Required layers

- Unit tests for pricing, quantities, transitions, permissions and state-machine invariants.
- Integration tests with real PostgreSQL/RabbitMQ in a controlled environment, including migrations, locks, outbox/inbox and constraints.
- Contract tests for REST and event schemas between services.
- End-to-end tests for web ordering and WhatsApp ordering, shopkeeper review, reservation, invoice, payment link and status delivery.
- Security tests for cross-tenant access, BOLA, privilege escalation, injection, CSRF where applicable, XSS, upload restrictions and webhook forgery/replay.
- Failure tests for provider timeouts, broker outage, consumer crash after commit, duplicate/out-of-order events, DB interruption, DLQ replay and graceful shutdown.
- Load tests for realistic traffic, bursts, slow dependencies, queue backlog and recovery; report environment and p50/p95/p99.

Release-blocking acceptance tests

- 100 concurrent attempts to reserve 10 available units never reserve more than 10 units.

- Repeated order/payment requests with the same idempotency key create one logical operation; reuse with a different payload is rejected.
- Concurrent duplicate webhooks cause one valid business transition and no duplicate payment/refund/invoice effect.
- Payment timeout remains uncertain until reconciliation; no blind second payment is created.
- Cancellation/payment-success races resolve according to documented policy and can be reconciled.
- RabbitMQ outage after local DB commit does not lose the outbox event; recovery eventually publishes it.
- A consumer crash/retry does not duplicate business effects beyond explicitly documented delivery semantics.
- Two-store tenant-isolation tests cover APIs, reports, invoices, payments, staff actions, direct service calls and webhook routing.
- Inventory balance/ledger reconciliation detects deliberately injected discrepancies.
- Invoice amounts and units match approved order snapshots, including fractional quantities, rounding and tax cases.
- A backup restores into an isolated environment and passes integrity checks.
- A failed deployment is recoverable using documented migration and rollback procedures.

16. Deployment, Backup and Recovery

- Maintain separate local, test/staging and production configurations. Never copy production secrets into development.
- Use pinned runtime/base-image versions, minimal containers, non-root execution where practical, resource limits and health checks.
- Use Docker Compose for local integration. Select production hosting based on availability, security, budget and operational skill.
- CI runs formatting/lint, type checking, unit/integration/contract tests as applicable, builds, migration validation, secret/dependency/image scanning.
- Apply migrations as a controlled step; verify backward compatibility and rollback constraints.
- Encrypt backups and define retention, access control, restore validation and off-site or provider-independent recovery where required.
- Define RPO (maximum acceptable data loss) and RTO (target recovery time) with the business owner, then prove them through drills.
- Document recovery of PostgreSQL, RabbitMQ durable messages, object/document storage, secrets and provider webhook processing.
- Use a controlled rollout, smoke/synthetic checks, dashboards, launch owner and rollback decision criteria.

17. Phased Implementation Plan

Phase	Deliverables / exit gate
0. Decisions	Approve PRD, ADRs, tax/payment/WhatsApp/hosting decisions, service boundaries, threat model, state machines, load/SLO assumptions.
1. Foundation	Repository, strict TypeScript, config validation, logging, IDs, OpenAPI, CI, Compose, migrations, health endpoints and tests.
2. Identity + Catalog	Store membership, roles, sessions, catalogue, variants, units, price history and tenant-scoped authorization tests.
3. Inventory	Movement ledger, atomic balances, reservation/release, expiry, adjustments, low-stock events; concurrency suite passes.
4. Orders + workflow	Review/revision, order snapshots, state machine, idempotency, saga/workflow persistence, outbox/inbox, cancellation and reconciliation.
5. WhatsApp + notification	Verified tenant mapping, inbound deduplication, conversation states, templates, delivery tracking and retry policies.
6. Billing + payments	Immutable invoice snapshots, provider sandbox adapter, signed webhooks, UNKNOWN-state reconciliation, refunds and tax-policy validation.
7. Reporting + UI	Staff/customer experiences, event-fed reports, audit trails, filters, accessible forms and failure states.
8. Hardening	Threat-model review, security/load/failure tests, backup restore drill, alerts, runbooks, migration and rollback rehearsal.
9. Controlled launch	Staging acceptance, operational owner, release/rollback plan, monitoring and post-launch review.

18. Risks and Required Mitigations

Risk	Required mitigation
Overselling / lost updates	Atomic reservations, row locks/conditional updates, constraints, stable lock order, concurrency tests.
Cross-service partial failure	Persisted workflow/saga, idempotent steps, compensations, reconciliation and stuck-state alerts.
Duplicate payment/refund	Provider/local idempotency, unique provider references/events, verified webhooks and reconciliation.
Lost or duplicate events	Transactional outbox/inbox, durable broker configuration, confirms, bounded retry and DLQ replay runbook.
Cross-tenant exposure	Identity-derived tenant scope, ownership checks, least privilege, optional RLS and regression tests.
Incorrect grocery totals	Canonical units, variant/SKU model, money precision, explicit rounding/tax policy and immutable snapshots.
WhatsApp ambiguity/outage	Conversation state machine, explicit confirmation, message dedupe, delivery tracking and manual recovery.
Unrecoverable deployment	Compatible migrations, verified backups, restore drills, rollback procedures and defined RPO/RTO.
Microservice operating burden	Limit independent deployments to meaningful boundaries; standardize telemetry, contracts, templates and ownership.

19. Architecture Decision Records (ADRs)

Create an ADR for each consequential choice. Minimum ADRs:

- ADR-001: Microservices boundaries and deployment model.
- ADR-002: PostgreSQL database/schema ownership and tenant isolation approach.
- ADR-003: Authentication/session model and customer identity linking.
- ADR-004: Payment provider, payment-link lifecycle, idempotency and reconciliation.
- ADR-005: Tax, invoice numbering, rounding, cancellation and credit-note policy.
- ADR-006: WhatsApp Cloud API, tenant mapping, templates, consent and conversation workflow.

- ADR-007: Order confirmation, inventory reservation timing/expiry and cancellation saga.
- ADR-008: Hosting, secret management, backup retention, RPO/RTO and SLO targets.
- ADR-009: Event schemas, compatibility, ordering and retry/DLQ policy.
- ADR-010: Product variants, units, fractional quantities, substitutions and price revisions.

Each ADR records context, options considered, decision, consequences, risks, owner, date and revisit triggers.

20. Definition of Done and Production Release Gate

- Every core requirement maps to an implementation component and one or more automated acceptance tests.
- All critical business invariants have documented owners and database/service enforcement.
- No unresolved critical/high security findings remain unless formally risk-accepted by the authorized owner.
- Inventory concurrency, tenant isolation, payment idempotency, webhook verification and workflow recovery tests pass.
- Outbox/inbox, retries, dead-letter replay and reconciliation have been tested under failure.
- Invoices and pricing pass agreed tax/accounting validation; customer-facing order and cancellation policies are documented.
- CI is green; migrations are reproducible; dependency, secret and container scans are reviewed.
- Dashboards and actionable alerts cover service health, queue backlogs, stuck workflows, payments, notifications and backups.
- A backup/restore drill passes; RPO/RTO and performance results are documented.
- Staging end-to-end acceptance passes; deployment, rollback, provider outage and incident runbooks are usable by another engineer.
- A named release owner approves launch and monitors post-deployment synthetic workflows.

Final Engineering Position

This document is a stronger, architecture-driven implementation baseline, not a mathematical guarantee of perfection. Correctness comes from implementing the specified invariants and proving them with code review, automated tests, security assessment, staging workflows, operational monitoring and recovery drills.

Do not begin live payment processing or production launch while mandatory provider/business decisions remain unresolved or release-blocking acceptance tests fail.